03. Collateral 수집 프로세스

Collateral 개요

Collateral은 Quote 검증에 필요한 부가 정보들의 집합으로, PCCS(Provisioning Certificate Caching Service) 서버나 Intel PCS(Provisioning Certification Service)에서 수집됩니다.

Collateral 구조

QuoteCollateralV3 구조

```
pub struct QuoteCollateralV3 {
pub pck_crl_issuer_chain: String, // PCK CRL 발급자 체인
pub root_ca_crl: Vec<u8>, // 루트 CA CRL
pub pck_crl: Vec<u8>, // PCK CRL
pub tcb_info_issuer_chain: String, // TCB 정보 발급자 체인
pub tcb_info: String, // TCB 정보 (JSON)
pub tcb_info_signature: Vec<u8>, // TCB 정보 서명
pub qe_identity_issuer_chain: String, // QE Identity 발급자 체인
pub qe_identity: String, // QE Identity (JSON)
pub qe_identity_signature: Vec<u8>, // QE Identity 서명
}
```

메인 Collateral 수집 함수

1. PCCS에서 Collateral 수집

```
pub  async  fn  get_collateral(pccs_url: &str, mut  quote: &[u8]) -> Result<Quot
let  quote = Quote::decode(&mut  quote)?;
let  ca = quote.ca().context("Failed to get CA")?;
let  fmspc = hex::encode_upper(quote.fmspc().context("Failed to get FMSPC")?);
get_collateral_for_fmspc(pccs_url, fmspc, ca, quote.header.is_sgx()).await
}
```

2. Intel PCS에서 Collateral 수집

```
pub  async  fn  get_collateral_from_pcs(quote: &[u8]) -> Result<QuoteCollateralV
get_collateral(PCS_URL, quote).await  // PCS_URL = "https://api.trustedservices.
}
```

PCCS 엔드포인트 관리

PcsEndpoints 구조체

```
struct PcsEndpoints {
base_url: String, // 기본 URL
```

```
tee: &'static str, // TEE 타입 ("sgx" 또는 "tdx")

fmspc: String, // FMSPC 값

ca: &'static str, // CA 타입 ("processor" 또는 "platform")

}
```

엔드포인트 생성 및 URL 정규화

```
impl PcsEndpoints {

fn  new(base_url: &str, for_sgx: bool, fmspc: String, ca: &'static str) -> Self

let  tee = if  for_sgx { "sgx" } else { "tdx" };

let  base_url = base_url

.trim_end_matches('/')

.trim_end_matches("/sgx/certification/v4")

.trim_end_matches("/tdx/certification/v4")

.to_owned();

Self { base_url, tee, fmspc, ca }

}

}
```

각종 엔드포인트 URL 생성

```
impl PcsEndpoints {

// PCK CRL 엔드포인트
```

```
fn  url_pckcrl(&self) -> String {
self.mk_url("sgx", &format!("pckcrl?ca={}&encoding=der", self.ca))
}
// 루트 CA CRL 엔드포인트
fn  url_rootcacrl(&self) -> String {
self.mk_url("sgx", "rootcacrl")
}
// TCB 정보 엔드포인트
fn  url_tcb(&self) -> String {
self.mk_url(self.tee, &format!("tcb?fmspc={}", self.fmspc))
}
// QE Identity 엔드포인트
fn  url_qe_identity(&self) -> String {
self.mk_url(self.tee, "qe/identity?update=standard")
}
// 공통 URL 생성 함수
fn  mk_url(&self, tee: &str, path: &str) -> String {
format!("{}/{}/certification/v4/{}", self.base_url, tee, path)
}
}
```

상세 Collateral 수집 프로세스

1. HTTP 클라이언트 설정

```
pub  async  fn  get_collateral_for_fmspc(
pccs_url: &str, fmspc: String, ca: &'static str, for_sgx: bool
) -> Result<QuoteCollateralV3> {
let  builder = reqwest::Client::builder();
#[cfg(not(feature = "js"))]
let  builder = builder
.danger_accept_invalid_certs(true) // 개발 환경용
.timeout(Duration::from_secs(180)); // 3분 타임아웃
let  client = builder.build()?;
let  endpoints = PcsEndpoints::new(pccs_url, for_sgx, fmspc, ca);
// ...
}
```

2. PCK CRL 수집

```
let  pck_crl_issuer_chain;
let  pck_crl;
{
let  response = client.get(endpoints.url_pckcrl()).send().await?;
pck_crl_issuer_chain = get_header(&response, "SGX-PCK-CRL-Issuer-Chain")?;
pck_crl = response.bytes().await?.to_vec();
```

```
};
```

3. TCB 정보 수집

```
let tcb_info_issuer_chain;
let raw_tcb_info;
{
let response = client.get(endpoints.url_tcb()).send().await?;
tcb_info_issuer_chain = get_header(&response, "SGX-TCB-Info-Issuer-Chain")
.or(get_header(&response, "TCB-Info-Issuer-Chain"))?;
raw_tcb_info = response.text().await?;
};
```

4. QE Identity 수집

```
let qe_identity_issuer_chain;
let raw_qe_identity;
{
let response = client.get(endpoints.url_qe_identity()).send().await?;
qe_identity_issuer_chain = get_header(&response, "SGX-Enclave-Identity-Issuer-Ch
raw_qe_identity = response.text().await?;
};
```

루트 CA CRL 수집 전략

1. PCCS 직접 수집 시도

```
let mut root_ca_crl = None;
if !endpoints.is_pcs() { // Intel PCS가 아닌 경우
root_ca_crl = http_get(&client, &endpoints.url_rootcacrl()).await.ok();
}
```

2. 인증서에서 CRL URL 추출

```
let root_ca_crl = match root_ca_crl {
Some(crl) => crl,
None => {
// QE Identity 발급자 체인에서 루트 인증서 추출
let certs = crate::utils::extract_certs(qe_identity_issuer_chain.as_bytes())?;
let root_cert_der = certs.last().unwrap();
// 루트 인증서에서 CRL Distribution Point URL 추출
let crl_url = extract_crl_url(root_cert_der)?;
let Some(url) = crl_url else {
bail!("Could not find CRL distribution point in root certificate");
};
// CRL URL에서 직접 다운로드
http_get(&client, &url).await?
```

```
}
};
```

CRL URL 추출 함수

```
fn extract_crl_url(cert_der: &[u8]) -> Result<Option<String>> {
let cert: Certificate = DerDecode::from_der(cert_der)?;
let Some(extensions) = &cert.tbs_certificate.extensions else {
return Ok(None);
};
for ext in extensions.iter() {
if ext.extn_id.to_string() != "2.5.29.31" { // CRL Distribution Points OID
continue;
}
let crl_dist_points: CrlDistributionPoints =
DerDecode::from_der(ext.extn_value.as_bytes())?;
for dist_point in crl_dist_points.0.iter() {
let Some(dist_point_name) = &dist_point.distribution_point else { continue; };
let DistributionPointName::FullName(general_names) = dist_point_name else { con
for general_name in general_names.iter() {
let GeneralName::UniformResourceIdentifier(uri) = general_name else { continue;
return Ok(Some(uri.to_string()));
}
```

```
}
}
Ok(None)
}
```

JSON 데이터 파싱

1. TCB 정보 파싱

```
let tcb_info_json: serde_json::Value =
serde_json::from_str(&raw_tcb_info).context("TCB Info should be valid JSON")?;

let tcb_info = tcb_info_json["tcbInfo"].to_string();
let tcb_info_signature = tcb_info_json
.get("signature")
.context("TCB Info missing 'signature' field")?
.as_str()
.context("TCB Info signature must be a string")?;
let tcb_info_signature = hex::decode(tcb_info_signature)
.ok()
.context("TCB Info signature must be valid hex")?;
```

2. QE Identity 파싱

```
let qe_identity_json: serde_json::Value =
serde_json::from_str(&raw_qe_identity).context("QE Identity should be valid JSON


let qe_identity = qe_identity_json
.get("enclaveIdentity")
.context("QE Identity missing 'enclaveIdentity' field")?
.to_string();
let qe_identity_signature = qe_identity_json
.get("signature")
.context("QE Identity missing 'signature' field")?
.as_str()
.context("QE Identity signature must be a string")?;
let qe_identity_signature = hex::decode(qe_identity_signature)
.ok()
.context("QE Identity signature must be valid hex")?;
```

HTTP 헤더 처리

헤더 추출 함수

```
fn get_header(response: &reqwest::Response, name: &str) -> Result<String> {
let value = response
```

```
.headers()
.get(name)
.ok_or_else(|| anyhow!("Missing {name}"))?
.to_str()?;
let value = urlencoding::decode(value)?; // URL 디코딩
Ok(value.into_owned())
}
```

통합 검증 함수

Collateral 수집 및 검증 원스톱 함수

```
pub async fn get_collateral_and_verify(
quote: &[u8],
pccs_url: Option<&str>,
) -> Result<VerifiedReport> {
let pccs_url = pccs_url.unwrap_or_default().trim();
let pccs_url = if pccs_url.is_empty() {
PCS_URL // Intel PCS 기본값
} else {
pccs_url
};
let collateral = get_collateral(pccs_url, quote).await?;
```

```
let now = SystemTime::now()
.duration_since(SystemTime::UNIX_EPOCH)?
.as_secs();
crate::verify::verify(quote, &collateral, now)
}
```

에러 처리 및 재시도 로직

HTTP 요청 헬퍼 함수

```
async fn http_get(client: &reqwest::Client, url: &str) -> Result<Vec<u8>> {
let response = client.get(url).send().await?;
if !response.status().is_success() {
bail!("Failed to fetch {url}: {}", response.status());
}
Ok(response.bytes().await?.to_vec())
}
```

이러한 Collateral 수집 프로세스를 통해 Quote 검증에 필요한 모든 부가 정보가 체계적으로 수집되어 검증 단계로 전달됩니다.